

Nakijk 02_01 — V1

NAKIJK V1 · Hoofdstuk 02 · Model, lijst en statuscodes · bij Oefening_02_01.md · eerste versie van je code, 9 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Controllers/ArtikelController.cs` en `Models/Artikel.cs`.

OPMERKING

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in.

Ik kon je API hier niet starten: op de werkruimte bij je laptop staat geen .NET, en het internet is daar afgesloten. Ik heb je code daarom gelezen en elk verzoek uit de opgave op papier nagelopen. Wil je het ook echt laten draaien, gebruik dan het nakijksript. Hoe, staat onder E.

In één oogopslag

Punt 1 tot en met 3 zijn in orde. Punt 4 werkt niet door één verkeerde route. Punt 5 werkt voor een soort die bestaat, maar niet voor een soort die niet bestaat. Punt 6 en 7 staan nog niet in je code: was je daar nog niet aan toe, lees ze dan als werk dat nog openstaat.

Punt	Wat de opgave vraagt	Status
1	het model <code>Artikel</code>	✓ in orde
2	de controller op <code>api/artikel</code> met minstens zes artikelen	✓ in orde
3	<code>GET api/artikel</code>	✓ in orde
4	<code>GET api/artikel/{id}</code>	✗ zie A1
5	<code>GET api/artikel/soort/{soort}</code>	½ zie A2
6	<code>GET api/artikel/voorradijg</code>	✗ ontbreekt, zie A3
7	<code>ArtikelFiche</code> en <code>GET api/artikel/fiche/{id}</code>	✗ ontbreekt, zie A4

Wat goed zit:

- Het model heeft alle zes de gegevens, met de juiste soort waarde. Tekst begint als lege tekst, zoals de les doet.
- Je lijst voldoet aan alle drie de eisen: zes artikelen, vijf soorten, twee artikelen met voorraad 0. Elektronica komt twee keer voor, en dat heb je straks nodig voor de fiche.
- De berichten bij een 404 staan letterlijk zoals in de opgave.
- De soortroute begint met `soort/`, en zit daardoor nooit in de weg van een andere route. Dat is waar de waarschuwing in punt 5 over gaat.

A. Wat een verkeerd antwoord geeft

A1. Punt 4: het endpoint luistert op een ander adres

Waar: `ArtikelController.cs`, het attribuut boven `GetArtikelId` (regel 25).

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/1</code>	404 Not Found, zonder bericht	200 OK met artikel 1
GET <code>api/artikel/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>

Waarom. De route in het attribuut van een methode komt áchter de route van de controller. Wat er nu in staat, komt dus achter `api/artikel`. Je endpoint luistert daardoor op `api/artikel/artikel/{id}`. Een verzoek naar `api/artikel/1` vindt geen enkel endpoint, en dan geeft ASP.NET zelf een lege 404. Je eigen bericht wordt nooit bereikt: daarom zie je twee keer hetzelfde antwoord, voor een artikel dat bestaat en voor een dat niet bestaat.

Richting.

- Open Scalar en kijk welk adres er bij deze methode staat. Leg het naast het adres van de soortroute: die doet het wel goed.
- Vraag je af welk stuk van de route de controller al levert, en welk stuk de methode er nog bij moet zetten.

A2. Punt 5: een soort die niet bestaat, geeft geen 404

Waar: `ArtikelController.cs`, de voorwaarde in `GetArtikelSoort` (regel 53).

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/soort/Elektronica</code>	200 OK met artikel 1 en 2	200 OK met de artikelen van die soort ✓
GET <code>api/artikel/soort/processor</code>	200 OK met []	404 Not Found met <i>We vonden geen artikelen van de soort processor.</i>

Waarom. Op regel 43 maak je de lijst `artikelen` zelf aan, nog vóór je zoekt. Die lijst bestaat dus altijd, ook als er geen enkel artikel in terechtkomt. De voorwaarde op regel 53 kan daardoor nooit waar zijn, en de 404 wordt nooit bereikt. Een lege lijst is niet hetzelfde als geen lijst.

Richting.

- Wat moet je aan een lijst vragen om te weten of er iets in zit? Je leerboek, hoofdstuk 6 (List), en de verwarring *Length*, *Count* en *Count()* achteraan zetten het op een rij.
- Dit is precies het leerdoel van de oefening: wanneer is leeg een fout (404), en wanneer een gewoon antwoord (200)? Hier vraagt de opgave een 404.

A3. Punt 6: het endpoint voor wat op voorraad is, ontbreekt

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/voorradi</code>	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met enkel de artikelen met een voorraad groter dan 0

Richting. Lees punt 6 samen met A2. Daar is een leeg resultaat een 404, hier niet: als er niets op voorraad is, vraagt de opgave een lege lijst met 200. Dezelfde situatie, een ander antwoord. Dat verschil maak je bewust per endpoint.

A4. Punt 7: de artikelfiche ontbreekt

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/fiche/1</code>	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met de fiche van artikel 1
GET <code>api/artikel/fiche/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>

Ook de klasse `ArtikelFiche` staat nog niet in de map `Models`.

Richting. Lees de twee zinnen onder de tabel van punt 7 twee keer. Ze leggen vast wat wel en niet meetelt: het artikel zelf telt niet mee bij `AantalVanDezelfdeSoort`, en `IsGoedkoopsteVanDeSoort` vergelijkt enkel met de andere artikelen van die soort.

B. Werkt, maar wijkt af

B1. `GetArtikelSoort` belooft één artikel, maar geeft een lijst

Waar: `ArtikelController.cs`, de kop van `GetArtikelSoort` (regel 40).

Het antwoord zelf klopt: de client krijgt een JSON-lijst. Maar het returntype zegt dat er één `Artikel` terugkomt. C# laat dat toe omdat `Ok(...)` alles aanvaardt, maar `Scalar` toont daardoor een verkeerd voorbeeld van het antwoord. Vergelijk het returntype met dat van `GetArtikelen`, dat ook een lijst teruggeeft.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
regel 29	Je build geeft hier een waarschuwing: <i>Converting null literal or possible null value to non-nullable type.</i> <code>FirstOrDefault</code> kan null geven, maar het type van je variabele zegt dat dat niet kan.	Kijk hoe de les die variabele schrijft, in de <code>ProductController</code> van je theorieproject, of in je leerboek, hoofdstuk 17.
regel 8 en 43	Het veld heet <code>Artikelen</code> , de lijst in <code>GetArtikelSoort</code> heet <code>artikelen</code> . Ze verschillen enkel in één hoofdletter, en dat maakt ze makkelijk te verwarren. Velden van een klasse beginnen in C# met een kleine letter; de les schrijft <code>laptops</code> .	Kies voor de lijst in de methode een naam die zegt wat erin zit.
<code>ProgrammingAdvancedOefeningen.http</code>	Het verwijst nog naar <code>/weatherforecast</code> , een overblijfsel van het sjabloon.	Weghalen of vervangen door je eigen verzoeken.

E. Zelf nakijken

Doe na het oplossen deze verzoeken opnieuw. Een fout is pas opgelost als het antwoord letterlijk overeenkomt met de opgave: statuscode én bericht.

Verzoek	Wat je moet zien
GET <code>api/artikel</code>	200 met je zes artikelen, zoals nu
GET <code>api/artikel/1</code>	200 met artikel 1
GET <code>api/artikel/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>
GET <code>api/artikel/soort/Elektronica</code>	200 met enkel artikelen van die soort
GET <code>api/artikel/soort/processor</code>	404 met <i>We vonden geen artikelen van de soort processor.</i>
GET <code>api/artikel/voorradijg</code>	200, en geen enkel artikel met voorraad 0 in het antwoord
idem, met tijdelijk alle voorraden op 0	200 met een lege lijst, geen 404
GET <code>api/artikel/fiche/{id}</code>	de drie controles onderaan punt 7 van de opgave
GET <code>api/artikel/fiche/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>

Met het nakijksript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_01 -Project  
"...\ProgrammingAdvancedOefeningen\ProgrammingAdvancedOefeningen"
```

Het script start je API zelf en stuurt de verzoeken uit de opgave. Het verslag komt in de map `Nakijken` naast je project.

Buiten deze oefening

Niet nagekeken, maar opgevallen: in de `OnderdelenController` heet de routeparameter van het codeendpoint anders dan de parameter van de methode. Dat is hetzelfde mechanisme als fout 1 tot 3 in 01_03.